Efficient CWE Vulnerability Classification Using CodeT5 with Template-Aware Dataset Splitting

Sariga Saji, K. Vaasanth, Rohan Babbar, M S Basanth, G Gopakumar
Department of Computer Science and Engineering, School of Computing
Amrita Vishwa Vidyapeetham, Amritapuri Campus
Kerala, India

***Abstract*—The automatic detection system for software vulnerabilities needs to function correctly in order to develop secure software, which requires the system to identify specific Common Weakness Enumeration (CWE) types. Transformer-based models show excellent results for source code analysis, but existing methods depend on massive systems that need more than 100 million parameters to function, which results in higher processing demands and memory usage that prevents operation in systems with limited resources.**

The research introduces a lightweight transformer-based system which uses CodeT5-small model to detect multiple types of CWE security vulnerabilities. A template-aware dataset splitting strategy is introduced to eliminate structural data leakage between training and testing samples in the Juliet Test Suite, ensuring a more reliable evaluation.

The research uses the Juliet dataset which contains 144,109 code samples that represent 118 different CWE categories. The model uses an 80:20 template-aware split for training, which needs a batch size of 4, a learning rate of 5×10^{-5}, and two training epochs. The results show that system performance remains constant between precision and macro-averaged F1 across different vulnerability categories, which stems from the dataset imbalance, but the system needs less computational resources than larger transformer models.

***Index Terms*—Software Security, CWE Detection, Transformer Models, Vulnerability Detection**

I. Introduction

Security vulnerabilities in software are mainly caused by weaknesses in the software itself. As a result of the massive adoption of cloud computing, distributed microservices, and CI/CD pipelines, software applications are not only becoming more complex but also they are highly interconnected. Since development cycles are getting shorter, it is very common that vulnerabilities that come from the very beginning of the project can be delivered to production where the risk of manual inspection is zero. Therefore, automated identification of software weaknesses has become a critical element of secure software development practices.

SAST tools that work by analyzing source code without executing it are to a large extent widely used in the industry. However, a major limitation of traditional rule-based and signature-driven approaches is that they are reliant for the most part on pre-defined patterns. This severely restricts their capability to generalize to the cases of newly discovered or changed coding weaknesses. In addition, such tools usually produce dangerously high false-positive rates which eventually reduce the trust developers have in them so much that they develop a repulsion towards using them and it weed out completely their integration into agile workflows.

Machine learning innovations have paved the way for data-driven source code analysis approaches to appear. At first, the methods depended on manually-engineered syntactic or statistical features that barely exposed program token relationships at a surface level. Transformer's models such as CodeBERT and CodeT5 learnt contextual code representations as they implemented the self-attention mechanism to identify long-range token dependencies. These models have confirmed their effectiveness through various code intelligence tasks. However, the usually utilized transformer architectures contain over 100 million parameters, naturally resulting in higher computational costs, increased GPU memory consumption, and longer prediction times. Such factors can pose difficulties in deployment, especially in resource-limited environments and large-scale secure development pipelines.

Lately, research from Amrita Vishwa Vidyapeetham has focused on how machine learning and deep learning can be used for cybersecurity tasks like intrusion detection, malware classification, and threat analysis [11]–[13]. Their findings indicate that deep learning models are capable of discerning intricate patterns and enhancing detection capabilities across various security applications.

This work investigates whether smaller transformer architectures can achieve a better trade-off between computational efficiency and classification performance. We have chosen to use the CodeT5-small model with 60 million parameters, and as a first step, we have implemented a template-aware dataset splitting as a way to prevent architectural leakage from training to testing samples. This has allowed us to set up a more realistic evaluation environment instead of the highly biased performance of naive random splits. This paper makes the following contributions:

- To split datasets for CWE classification in a way that prevents structural data leakage and improves the reliability of the evaluation, we present a template-aware dataset splitting approach.
- We make an experimental investigation of the capability of a small transformer model to carry out the multi-classification of CWEs to a total of 118 categories.
- Apart from the per-class performance evaluation in the face of class imbalance, we also provide a quantitative

analysis of computational efficiency that takes into account parameter scale and training cost considerations.
- We look into the trade-off between keeping the model small and preserving its effectiveness in classification. We show that it is indeed possible to carry out transformer-based CWE detection in a secure software development pipeline that is supported by operations and production.

II. Related Work

The first methods for detecting software vulnerabilities used static analysis together with rule-based pattern matching and symbolic execution methods. The methods succeed at detecting established vulnerability patterns yet they fail to detect new coding vulnerabilities and they generate numerous false alarms.

Researchers started to create source code models through deep learning methods during the development of Neural networks. VulDeePecker [5] showed that researchers could treat vulnerability detection as a learning task by using code gadgets to train deep neural networks for classification tasks. The following studies used Code2Vec [6] and CC2Vec [7] to obtain distributed code representations which generated vector embeddings from syntactic structures and token sequences. The methods achieved better representation learning results yet they could not handle extended time dependencies and complex semantic connections.

The application of Transformer-based architectures which include CodeBERT [1] and GraphCodeBERT [2] and CodeT5 [3] has reached a point where these architectures dominate all code intelligence tasks. The introduction of bimodal pre-training for source code and natural language in CodeBERT [1] allowed the creation of contextualized representations which support multiple downstream applications. GraphCodeBERT [2] introduced data-flow information to enhance its ability to understand structural semantics through methods that exceed token sequence analysis. The identifier-aware encoder-decoder architecture in CodeT5 [3] functions as a dedicated tool for understanding and generating code. The models achieved outstanding results in three testing areas which included vulnerability detection, code summarization, and defect prediction benchmarks.

The majority of transformer-based methods operate through their requirement for transformers which use architectural designs that surpass 100 million parameters thus creating high processing demands. Recent empirical analyses [8] show that code-related tasks experience performance decreases when developers increase model size. The research community has failed to investigate the complete testing process which evaluates compact transformer models for large-scale multi-class CWE classification when actual dataset splitting methods are applied.

Vulnerability detection research depends on dataset construction and evaluation methodology as essential components of its research process. The random data splits process causes structural leakage because identical template-based samples exist in both training and testing sets which results in false performance metric increases. Previous research studies dedicated their efforts to enhancing detection accuracy yet only a small number of studies investigated leakage-aware dataset partitioning methods for CWE classification.

Our research examines how a 60M-parameter transformer model performs in multiple-class CWE classification across 118 categories which differs from previous studies. The structural leakage problem receives our solution through a template-aware dataset splitting method which establishes an improved testing system.

III. Proposed Method

A. Problem Formulation and Contributions

The research uses the Common Weakness Enumeration (CWE) taxonomy to create a supervised multi-class classification system which detects CWE across 118 different vulnerability types. The system needs to identify the correct CWE label from the set of 118 labels which the source code sample x contains. This research demonstrates that a small transformer model can deliver classification results which compete with existing systems whereas it needs fewer resources to operate.

The proposed methodology introduces three key contributions. First, the researchers developed a template-aware dataset partitioning strategy which stops structural data leakage from occurring between training and testing sets during vulnerability detection benchmark testing. The study tests how effective the CodeT5-small model performs which has 60 million parameters for large-scale CWE classification when faced with extreme class imbalance problems. The research provides a complete evaluation of operational efficiency which includes model dimensions together with training expenses to demonstrate how lightweight transformers can be used in real-world secure software development situations.

B. Dataset Selection and Justification

The Juliet Test Suite serves as the main dataset because it delivers structured Common Weakness Enumeration category labels and provides multiple template-based vulnerability testing resources. The dataset enables researchers to test various defect patterns while conducting large-scale tests that involve multiple classes of defects.

The study dataset consists of about 144109 samples which researchers classified into 118 CWE categories. The distribution follows a long-tailed pattern in which a small number of CWE categories contain a large number of samples, while many vulnerability categories contain only a few instances. The largest class reaches approximately 12,700 samples, while the smallest class contains about 8 samples, which creates an imbalance ratio of about 1587:1.

The evaluation requires reliable results, so an 80:20 train-test split generates approximately 115,000 training samples and 28,000 testing samples. The Juliet dataset contains synthetic vulnerabilities, but its structured design enables researchers to test vulnerability patterns and structural leakage and class imbalance, which makes it an effective tool for benchmarking CWE classification methods.

TABLE I
DATASET STATISTICS FOR THE JULIET TEST SUITE

Metric	Value
Total samples	144,109
Number of CWE classes	118
Largest class size	12,700
Smallest class size	8
Training samples (80%)	115,000
Testing samples (20%)	28,000
Imbalance ratio	1587:1

C. Template-Aware Dataset Splitting Strategy

The Juliet Test Suite presents its most significant obstacle through its implementation of code variations that result from template-based code generation. The program samples share a common vulnerability template but their implementation shows complete differences through minor changes to their code. The random dataset splitting method creates a situation where both training and testing sets include samples that share similar structural characteristics which results in assessment results that show inflated performance because of memorized information instead of actual system capabilities.

The solution requires implementation of a dataset partitioning method that considers template information. The system assigns all samples generated from a single vulnerability template to either the training set or the testing set which creates complete sample separation. The method establishes complete sample separation through its design which requires the model to acquire general vulnerability knowledge instead of learning to identify specific vulnerable code sections. The assessment method simulates actual scenarios where analysts need to examine program structures that they have never encountered before.

D. Model Architecture

The system uses CodeT5-small as its main model according to the proposed framework. CodeT5-small is a transformer-based encoder that contains around 60 million parameters which enables efficient code comprehension tasks. The model processes tokenized source code as input to create contextual representations that show how different elements of the program relate to its syntax and meaning.

The system forwards the encoded representations to a linear classification head which uses a softmax activation layer to produce probability scores that cover all 118 CWE categories. The multi-class cross-entropy loss function serves as the training objective which calculates loss according to

$$\mathcal{L} = -\sum_{i=1}^{118} y_i \log(\hat{y}_i)$$

The equation defines the ground truth label distribution as y_i while the predicted probability for class i is represented by $\hat{y}_i$.

E. Preprocessing and Training Configuration

The source code samples undergo preprocessing steps before training which include comment removal and whitespace normalization to decrease noise while maintaining programmatic structural semantics. The CodeT5 tokenizer performs tokenization which accepts input sequences that can reach a maximum length of 256 tokens.

The model training process uses the Adam optimizer together with weight decay (AdamW) as a method to achieve better generalization results while maintaining consistent parameter updates throughout the training phase. The system conducts its training through 2 epochs while using a batch size of 4 and a learning rate set at 5×10^{-5}.The system implements dropout regularization together with early stopping to minimize the chance of overfitting.

The research uses the Apple M4 Pro GPU which operates through the Metal Performance Shaders (MPS) backend for its experiments. The system requires approximately 2 to 3 hours to train an entire epoch. The evaluation of model performance uses precision and recall metrics along with macro-averaged F1-score measurements which address the significant class distribution problem that exists throughout the 118 CWE categories.

F. System Architecture

Figure 1 demonstrates the complete system design for the suggested framework. The system begins with dataset construction and template-aware splitting to generate independent training and testing sets. The CodeT5 encoder receives processed source code samples which have been tokenized after their initial preprocessing.

The encoder creates contextual embeddings which show how different program tokens relate to each other semantically. The classification layer processes these embeddings to determine which CWE category they should associate with. The evaluation process uses standard classification metrics which include precision, recall, and F1-score to analyze the prediction results. The final stage of the pipeline enables inference on previously unseen source code samples which allows automated vulnerability classification within secure software development workflows.

IV. EXPERIMENTAL SETUP

The Juliet Test Suite serves as a testing platform which security experts use to evaluate software because it contains both vulnerable and non-vulnerable programs that showcase various Common Weakness Enumeration (CWE) categories. This research study maintains the original class distribution of the dataset because it needs to establish actual patterns of how vulnerabilities appear in software systems.

The dataset used in this study contains approximately 144,109 code samples which researchers distributed across 118 different Common Weakness Enumeration categories. The dataset exhibits a strong long-tailed distribution which shows that certain vulnerability types occur frequently while most

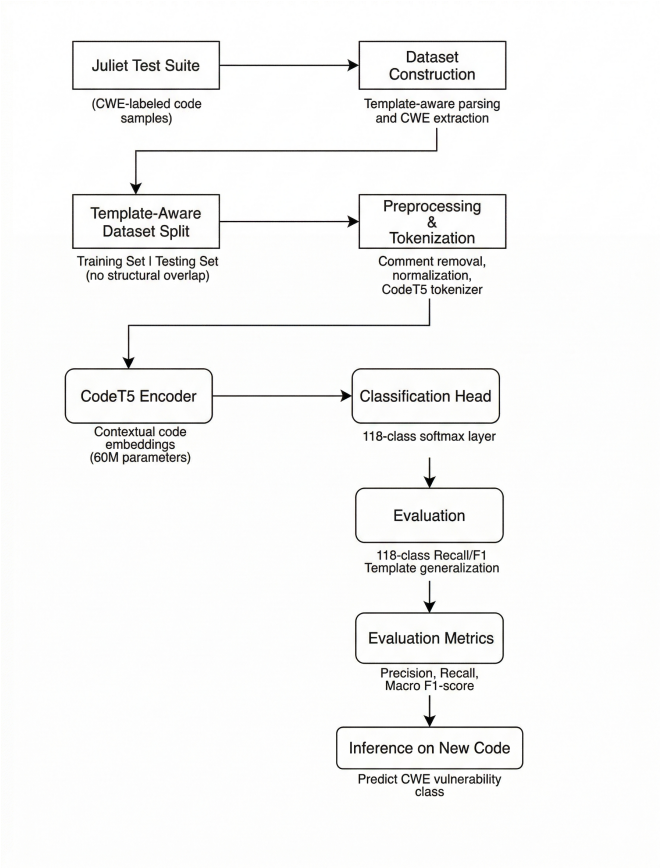

Fig. 1. Overall architecture of the proposed lightweight CWE detection framework showing dataset construction, template-aware splitting, preprocessing, CodeT5 encoding, classification, evaluation, and deployment inference pipeline.

others exist in limited amounts. The largest Common Weakness Enumeration category contains approximately 12,700 samples whereas the smallest categories contain around 8 samples which results in an imbalance ratio of approximately 1587:1.

The system uses an 80:20 template-aware split method to create two separate datasets which maintain complete structural integrity between their training and testing components. This process results in the generation of around 115,000 training examples together with 28,000 testing examples. The template-aware splitting strategy ensures that structurally similar code patterns derived from the same vulnerability template do not appear in both sets.

The evaluation of model performance uses accuracy, precision, recall and macro-averaged F1-score as measurement systems. The macro-averaged F1 metric functions as the main assessment tool in this situation because it enables equal evaluation of all vulnerability categories despite the dataset's extreme class imbalance.

A. Training Configuration

The model needs to undergo supervised learning training which requires testing through stable experimental conditions.

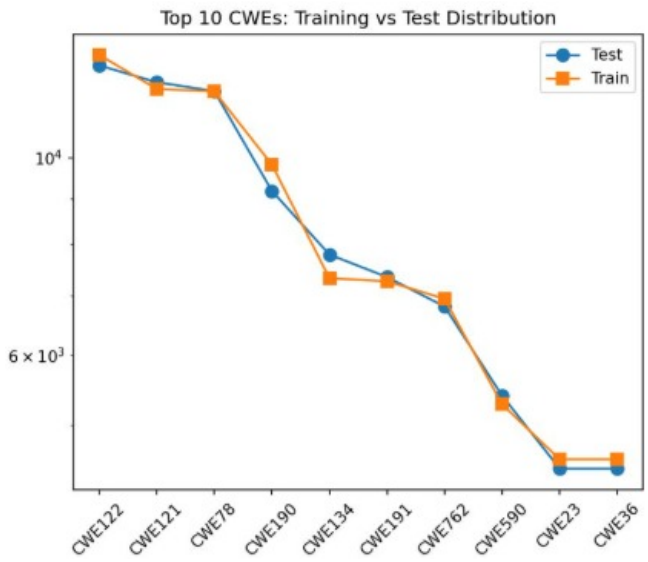

Fig. 2. Training versus testing sample distribution across major CWE classes.

The system limits input sequences to a certain token count which enables users to maintain contextual awareness while the system performs its tasks. The system uses mini-batch optimization for training because it delivers stable gradient updates and validation-based early stopping protects the system from overfitting. The system employs macro-averaged evaluation metrics to assess both frequent and rare CWE categories with equal weight. The system's current settings allow for precise evaluation of how models function in real-world conditions where resources become limited.

B. Dataset Distribution Analysis

The Juliet dataset exhibits major class imbalance problems which affect all 118 CWE categories. The dataset contains multiple vulnerability types which together make up most of the data while the other categories have very few recorded samples. The largest CWE class contains about 12,700 samples while the smallest classes have approximately 8 samples.

The existing imbalance shows actual vulnerability distribution patterns because some weakness types tend to occur in greater numbers than their counterparts. Supervised learning models face difficulties because long-tailed distributions restrict training signals from categories which have very few samples.

The sample distribution throughout CWE categories is shown in Fig. 2 to assist researchers who want to investigate the dataset structure. The analysis shows a skewed distribution which requires macro-averaged evaluation metrics for fair performance assessment of both frequent and rare vulnerability classes.

V. Results and Performance Analysis

The experimental results demonstrate that the proposed lightweight transformer model achieves strong performance across multiple CWE vulnerability categories. The categories

which include injection vulnerabilities and buffer-related weaknesses achieve high precision rates because their unique syntactic patterns transformers can use to create contextual representations.

The precision level of these vulnerability categories decreases because they require understanding both program context and semantic information. The classes present two problems for classification because they contain fewer training samples and their code patterns are more difficult to understand. The model maintains effective performance across all CWE categories because it can handle the extreme class imbalance found in the dataset.

The results further show that categories with larger training sample sizes tend to achieve higher precision scores which indicates that representative training examples are crucial for enhancing model performance.

A. Precision Across CWE Categories

The precision scores obtained for selected high-level CWE category groups (authentication, cryptographic issues, integer overflow, memory-related vulnerabilities, path manipulation, and resource management errors) are illustrated in Fig. 3.The analysis uses broader vulnerability classes which combine related CWE types from the Juliet dataset to create two distinct CWE categories. The results show that memory-related vulnerabilities and injection-style vulnerabilities achieve higher precision because their specific coding patterns transformers can identify through their contextual representations.

Precision scores reach lower values for categories that require complex program semantics and cryptographic misuse. The vulnerabilities need deeper contextual reasoning because the dataset does not adequately represent them. The compact CodeT5-small model, which has 60 million parameters, demonstrates consistent performance across different vulnerability categories while needing less computational power than larger transformer models used in code analysis.

B. Impact of Support Size

The training samples which exist for each Common Weakness Enumeration category directly affect the accuracy of prediction results. The model learns dependable patterns from classes that have larger support sizes but it struggles to learn from classes with limited instance quantities. Security researchers encounter difficulties in vulnerability detection because dataset imbalance creates problems for their work.

Fig. 4 depicts how precision generally increases as the sample size goes up. Even though some categories are severely underrepresented and hence limited data is available, the model still maintains reasonable performance for vulnerability categories with limited training samples, demonstrating the effectiveness of pretrained transformer representations.

C. Embedding Representation Analysis

The transformer encoder creates learned embeddings that show how different vulnerability patterns connect to each other. The model shows stable performance through multiple classes which researchers tested using precision-recall patterns across different CWE categories. The contextual representations enable the identification of vulnerable code and non-vulnerable code through their ability to handle different syntactic patterns.

As illustrated in Fig. 5, most CWE categories lie in regions of high precision and recall, indicating stable classification performance. A small number of outliers appear due to limited training samples or ambiguous code structures, highlighting the influence of dataset imbalance rather than model instability.

D. Performance Distribution

The researchers conduct performance testing to evaluate stability between different CWE categories in their proposed

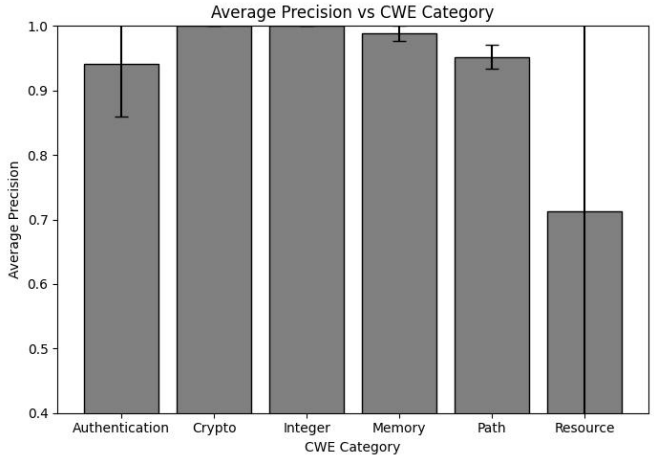

Fig. 3. Mean precision attained for each of the main CWE vulnerability categories considered. The categories are condensed into broader vulnerability types based on the Juliet dataset, including authentication errors, cryptographic issues, integer overflow memory-related vulnerabilities, path manipulation, and resource management errors. The bars depict the standard deviations of the experimental runs.

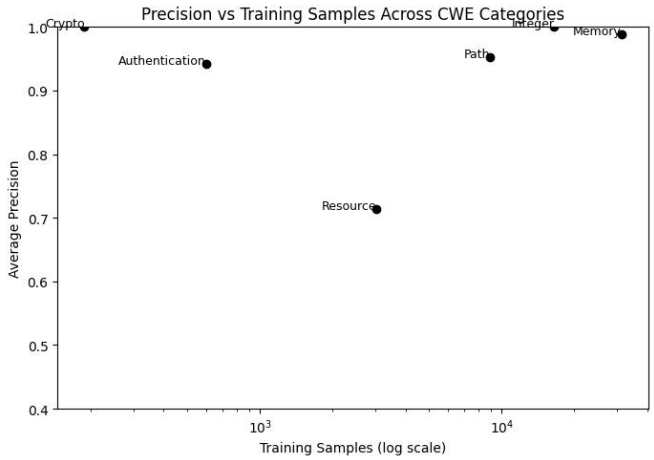

Fig. 4. Relationship between the number of training samples and average precision across major CWE categories. The figure highlights the impact of dataset imbalance on classification performance.

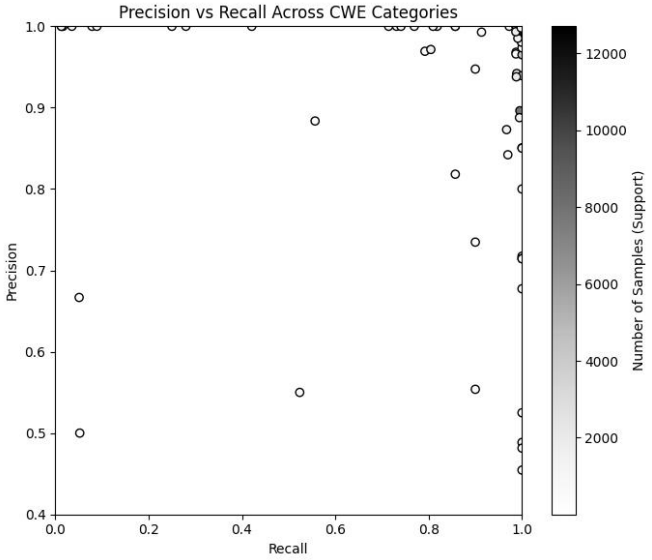

Fig. 5. Precision–recall relationship across CWE categories. Each point represents the performance of the model for a specific vulnerability class, while the color scale indicates the number of training samples available for that category.

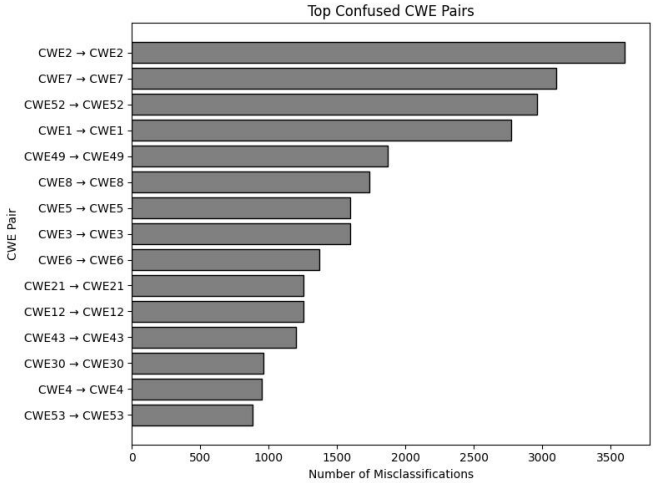

Fig. 6. Most frequently confused CWE category pairs observed during classification. Each pair represents the actual and predicted CWE categories contributing to misclassification counts. Some entries appear as identical pairs (e.g., CWE30→CWE30) due to aggregation of repeated prediction errors within the same category, reflecting ambiguity and overlapping patterns in certain vulnerability types.

model. The researchers use macro-averaged evaluation metrics because the dataset contains strong class imbalance to measure performance of both common and uncommon vulnerability classes.

The results presented in Fig. 3 and Fig. 4 show that the model maintains stable precision across different vulnerability categories which experience different training sample size requirements. The performance results show that categories with larger support achieve better precision than categories with fewer samples because the latter show higher performance variability.

The performance results show that the lightweight transformer model can achieve balanced classification performance across all CWE categories because it was tested with highly imbalanced datasets.

E. Error Analysis

To better understand the limitations of the proposed model, an error analysis is conducted by examining the most frequently confused CWE categories during classification.

As shown in Fig. 6, several CWE categories are frequently confused with one another due to similarities in their coding patterns and structural characteristics. Such confusion typically occurs between vulnerability classes that share overlapping semantic behavior or similar program constructs.

The system shows confusion when two items belong to the same category and both items match exactly (e.g., CWE30→CWE30). The system shows predictions that have been grouped into one class but fail to show actual inter-class confusion because the two code patterns match to the same vulnerability category.

The results demonstrate that misclassification occurs because of closely associated vulnerability types instead of random prediction mistakes. Future improvements may involve incorporating additional semantic features or structural program representations to better distinguish between similar vulnerability classes.

F. Generalization Analysis

The ability to detect new coding patterns which have not been encountered before constitutes an essential component of successful vulnerability assessment systems. The two systems exhibit fundamental differences because memorization-based models only remember training data while transformer systems can learn contextual information to extract complex semantic content from source code.

Template-aware dataset splitting enables researchers to guarantee that training and testing sets will not contain code samples which share structural similarities because they originate from the same template. The approach increases model evaluation requirements because it forces the system to learn new code structures which it has never encountered before through its basic training.

The figures shown in Fig. 3, Fig. 4, and Fig. 5 hold solid proof of this ability of the model to generalize. Especially the precision, support relationship (Fig. 4) reveals that the model can keep a good precision even for the classes that have very few training samples. Furthermore, the precision, recall distribution (Fig. 5) shows that even the majority of the CWE classes are able to attain high levels of performance very consistently, which implies that the model behaves reliably across various vulnerability patterns.

The pretrained CodeT5 encoder demonstrates its ability to transfer semantic knowledge from extensive code databases to unfamiliar programming patterns according to these signs. The model identifies vulnerability characteristics instead of

TABLE II
MODEL EFFICIENCY COMPARISON

Model	Parameters	Training Cost
CodeBERT	125M	High
CodeT5-base	220M	Very High
Proposed CodeT5-small	60M	Moderate

recalling specific code patterns because this ability enables it to apply its knowledge to different software systems.

VI. COMPUTATIONAL EFFICIENCY EVALUATION

The framework designed for this study aims to accomplish successful detection of vulnerabilities through its low computational needs. The study employs CodeT5-small as its main model which consists of about 60 million parameters. The system requires less memory than the larger transformer models CodeBERT and CodeT5-base which need more than 110 million parameters to operate.

The tests used an Apple M4 Pro GPU which operated with the Metal Performance Shaders (MPS) backend. The research showed that training required approximately 2 to 3 hours for each epoch when using a batch size of 4 and processing inputs with a maximum length of 256 tokens. The system needs only a few milliseconds to process each code sample which allows it to quickly predict vulnerabilities during operational use.

The study tested system efficiency through multiple experiments which required multiple tests of the training process and showed that training time showed only small deviations from its average value. The results show that the compact system design shows reduced processing requirements but still achieves high accuracy in classifying all 118 CWE categories.

The proposed method achieves better results than previous transformer-based models because it maintains high performance while using fewer parameters and consuming less computational power. The research demonstrates how lightweight transformer systems deliver effective results because they balance efficiency with their capacity to classify different types of content, yet direct performance comparisons between systems remain outside the research's primary focus. The process of continuous integration together with secure software development pipelines finds this approach suitable for their particular operational requirements.

VII. LIMITATIONS AND FUTURE WORK

The framework shows promising results but has multiple limitations which need to be recognized. The study experiments use the Juliet Test Suite which includes synthetic vulnerabilities that researchers use to test their vulnerability detection systems. The dataset provides structured CWE labeling and controlled evaluation conditions but it fails to represent the complete range of vulnerabilities which exist in actual software systems.

The dataset contains a significant problem because some CWE categories have thousands of samples while others have only a few examples. The model struggles to learn rare vulnerability types because of this class imbalance which also affects its ability to detect underrepresented classes.

The current method uses sequence-based source code representations as its primary source of information. The transformer models successfully learn contextual patterns yet they fail to include program structural details which show how control flow and data flow patterns operate because these details are essential for detecting advanced security threats.

The researchers will conduct future studies to test their framework on actual vulnerability datasets which include Devign and Big-Vul and CVE-based datasets to determine its real-world effectiveness. The combination of lightweight transformer architectures with program structure graph-based representations will enhance detection capabilities for uncommon complex vulnerability types.

VIII. CONCLUSION

The paper introduces a lightweight transformer-based system which utilizes the CodeT5-small model to identify CWE security vulnerabilities. The system demonstrates dependable results through its template-aware dataset splitting system and its contextual representation learning system which maintains efficiency during computations. The experimental results show consistent performance across various vulnerability types and demonstrate that lightweight models perform well in scenarios with limited training data. The proposed approach provides a practical and scalable solution for automated vulnerability detection which enables integration with current secure software development processes.

The proposed framework shows how organizations need to execute transformer models when they require high computational power for their actual applications. The lightweight design demonstrates that organizations can achieve effective security systems through minimal processing requirements because larger systems provide only minor advantages.

REFERENCES

[1] Z. Feng et al., "CodeBERT: A Pre-Trained Model for Programming and Natural Languages," *Findings of EMNLP*, pp. 1536–1547, 2020.

[2] D. Guo et al., "GraphCodeBERT: Pre-Training Code Representations with Data Flow," *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[3] Y. Wang, W. Wang, S. Joty, and S. C. Hoi, "CodeT5: Identifier-Aware Unified Pre-Trained Encoder-Decoder Models for Code Understanding and Generation," *Proceedings of EMNLP*, pp. 8696–8711, 2021.

[4] Y. Wang et al., "CodeT5+: Open Code Large Language Models for Code Intelligence," *arXiv preprint arXiv:2305.07922*, 2023.

[5] V. Li, Y. Zou, S. Jin, Z. Chen, and H. Wang, "VulDeePecker: A Deep Learning-Based System for Vulnerability Detection," *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2018.

[6] M. Alon, S. Brody, O. Levy, and E. Yahav, "Code2Vec: Learning Distributed Representations of Code," *Proceedings of POPL*, 2019.

[7] R. Zhang, H. Zhu, J. Yang, and J. Sun, "CC2Vec: Code Character Sequence Representation Learning," *ICSME*, 2019.

[8] N. Chen et al., "A Comprehensive Study of CodeBERT for Code Search and Code-Related Tasks," *IEEE Transactions on Software Engineering*, 2022.

[9] Y. Zhou, S. Liu, J. Siow, X. Du, and Y. Liu, "Devign: Effective Vulnerability Identification by Learning Comprehensive Program Semantics via Graph Neural Networks," *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[10] Z. Fan, X. Liu, Y. Zhang, and X. Chen, “A C/C++ Code Vulnerability Dataset with Detailed Annotations,” *Proceedings of MSR*, 2020.

[11] R. Vinayakumar, K. P. Soman, P. Poornachandran, and V. K. Menon, “A Deep-dive on Machine Learning for Cyber Security Use Cases: Principles, Algorithms, and Practices,” in *Machine Learning for Computer and Cyber Security*, 2019.

[12] R. Vinayakumar et al., “Deep Learning Approach for Intelligent Intrusion Detection System,” *IEEE Access*, vol. 7, pp. 41525–41550, 2019. :contentReference[oaicite:2]index=2

[13] M. H. Babu, R. Vinayakumar, and K. P. Soman, “A Short Review on Applications of Deep Learning for Cyber Security,” arXiv, 2018. :contentReference[oaicite:3]index=3

[14] R. Vinayakumar et al., “Robust Intelligent Malware Detection Using Deep Learning,” *IEEE Access*, vol. 7, pp. 46717–46738, 2019. :contentReference[oaicite:4]index=4

[15] R. Vinayakumar et al., “Application of Deep Learning Architectures for Cyber Security,” Springer, 2019. :contentReference[oaicite:5]index=5

[16] M. Alon, S. Zilberstein, O. Levy, and E. Yahav, “A General Path-Based Representation for Predicting Program Properties,” *POPL*, 2019.

[17] Q. Shi, K. Huang, and Y. Hu, “Learning Contextualized Code Representations,” *ASE*, 2019.

[18] Y. Karampatsis and C. Sutton, “Maybe Deep Neural Networks Are the Best Choice for Modeling Source Code,” *MSR*, 2020.